Solution: Problem Challenge 2: Path with Maximum Sum

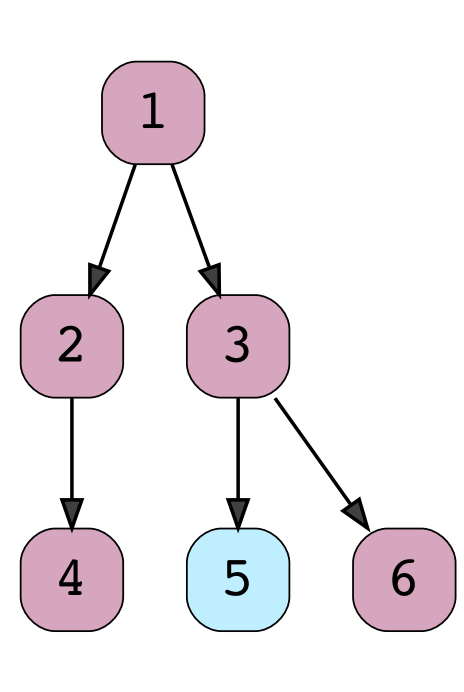
Problem Statement

Find the path with the maximum sum in a given binary tree. Write a function that returns the maximum sum.

A path can be defined as a **sequence of nodes between any two nodes** and doesn't necessarily pass through the root. The path must contain at least one node.

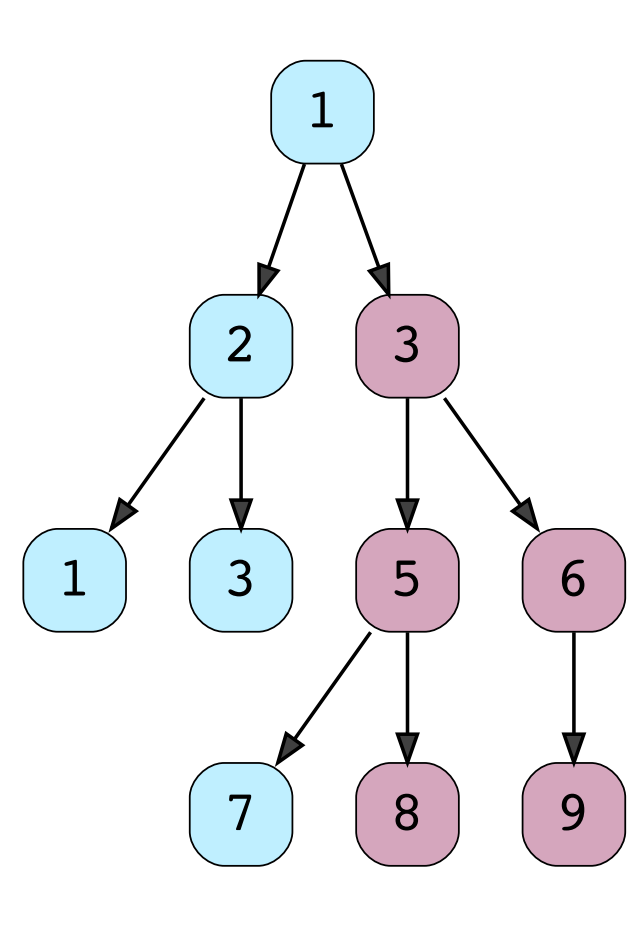
Example 1:

Output: 16
Explanation: The path with maximum sum is: [4, 2, 1, 3, 6]



Example 2:

Output: 31
Explanation: The path with maximum sum is: [8, 5, 3, 6, 9]

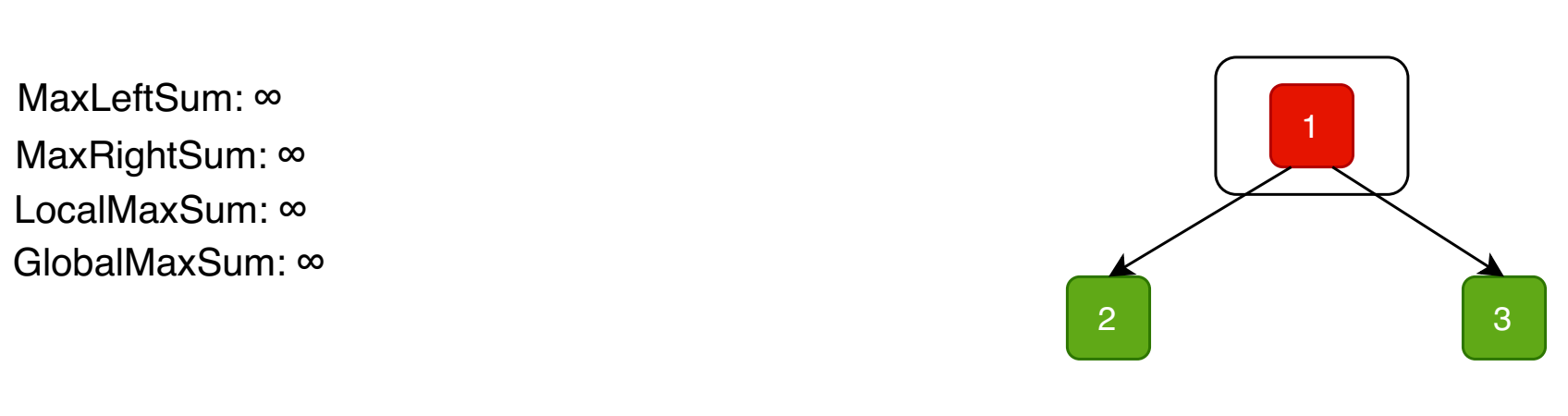


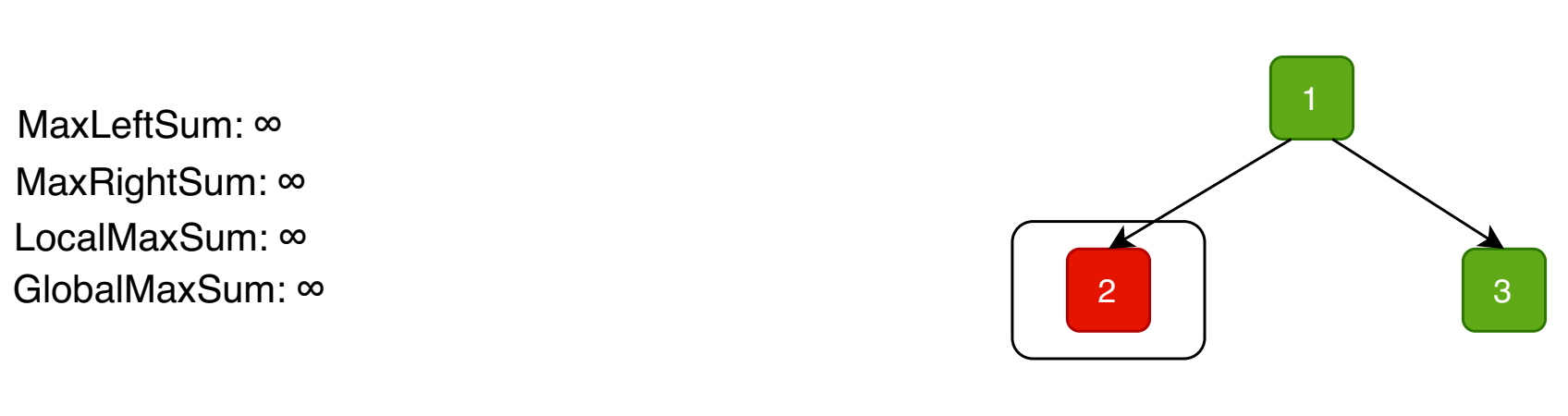
Solution

This problem follows the **Binary Tree Path Sum** pattern and shares the algorithmic logic with **Tree Diameter.** We can follow the same **DFS** approach. The only difference will be to ignore the paths with negative sums. Since we need to find the overall maximum sum, we should ignore any path which has an overall negative sum.

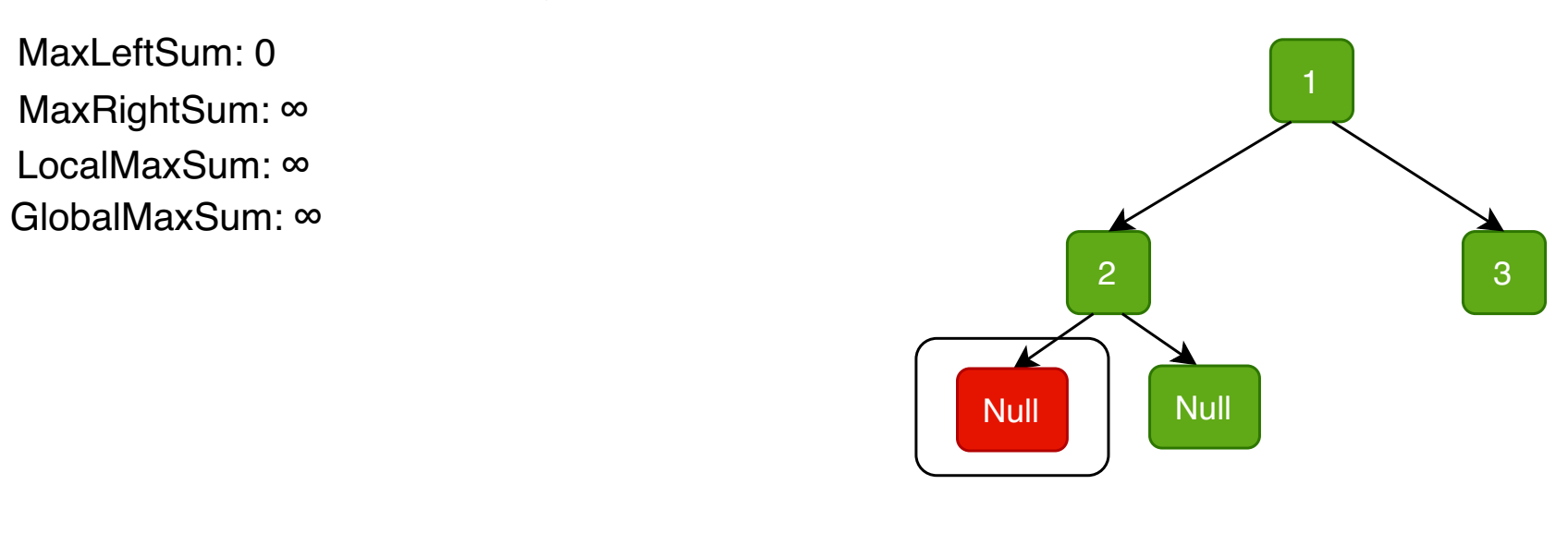
Here is the visual representation of the algorithm:

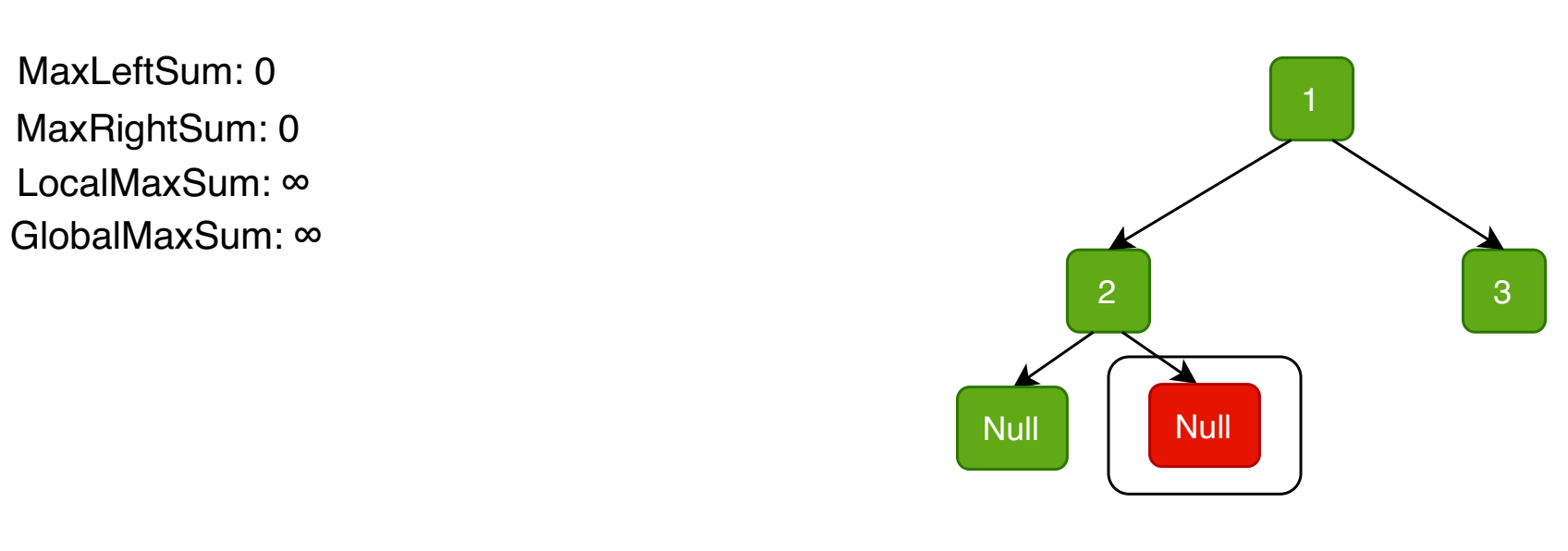
Example: 1

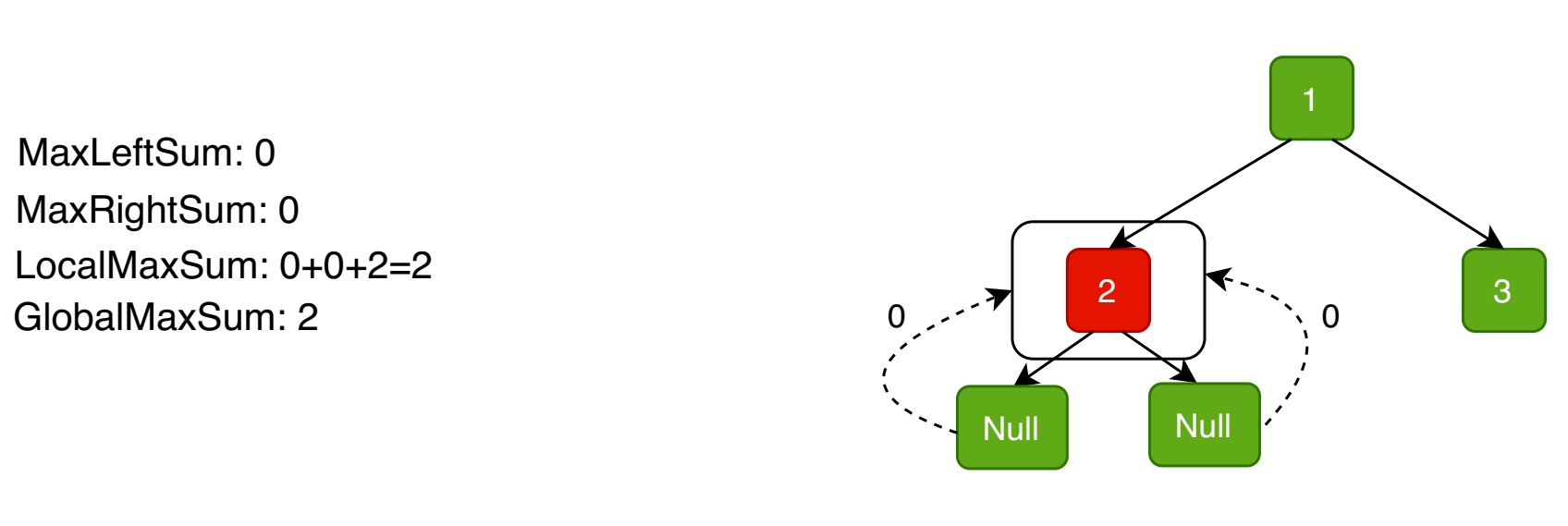




Since exit condition is called so the method will return 0 and hence we will move to the right of the node 2

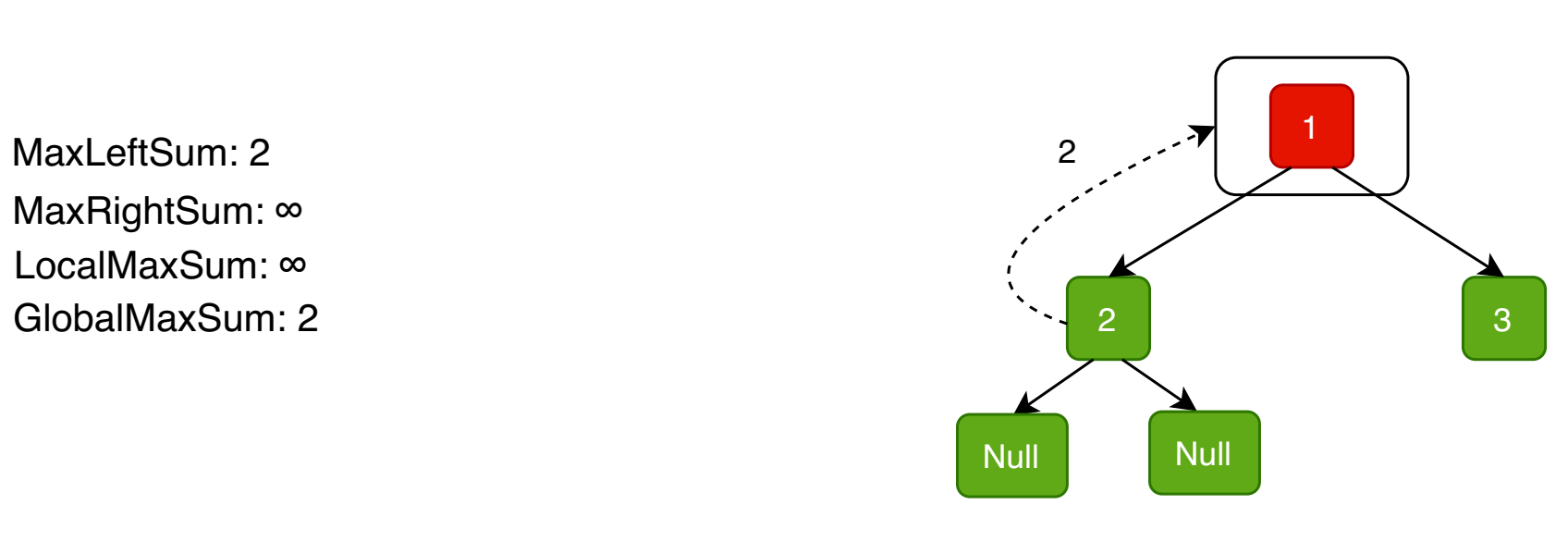


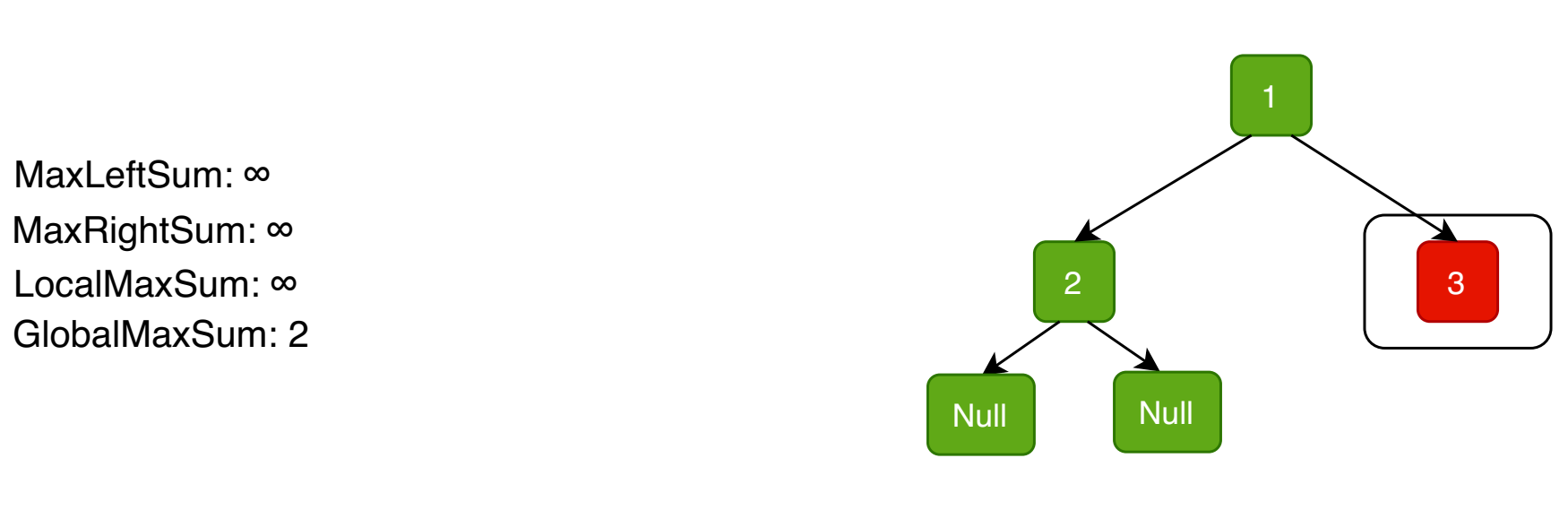




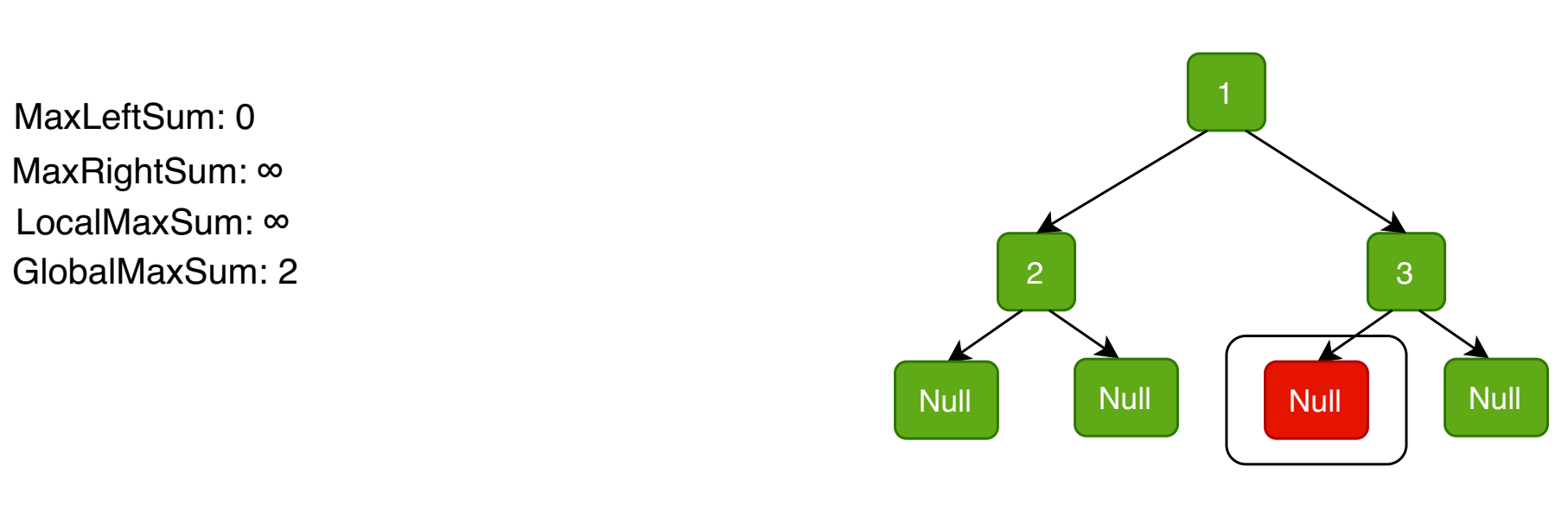
Now, while backtracking, we will return the maximum path sum to the respective roots and when we backtrack to 1, the max. path sum is 2 as the left and right are null for node 2

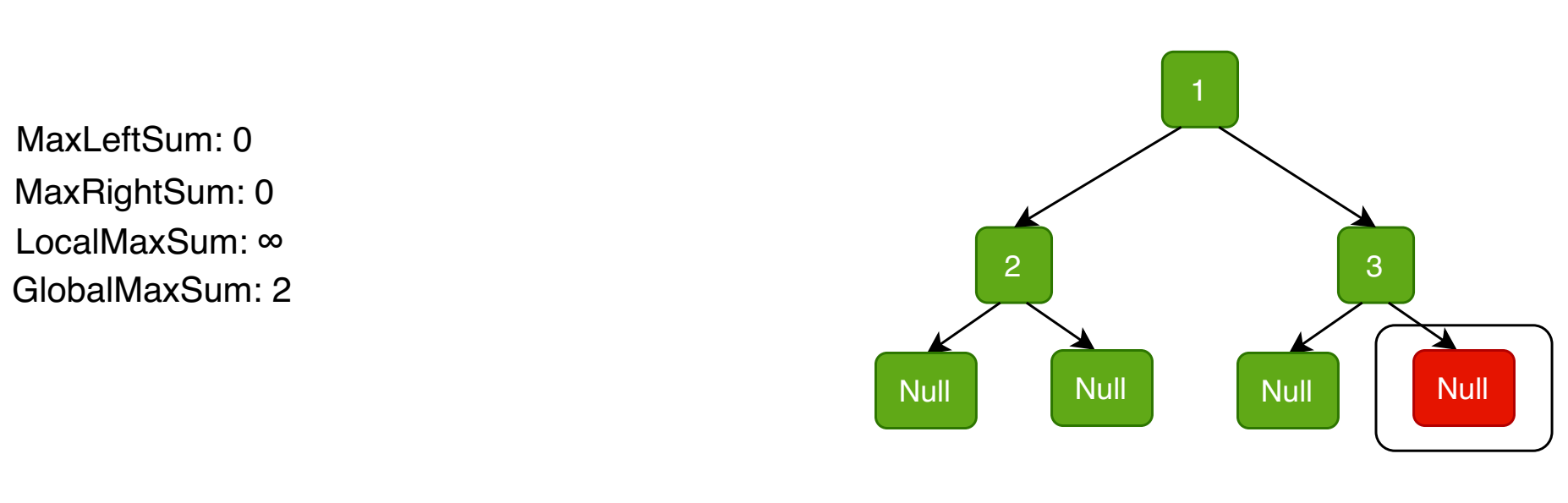
Since we have covered the left path so now we will move to right of the root

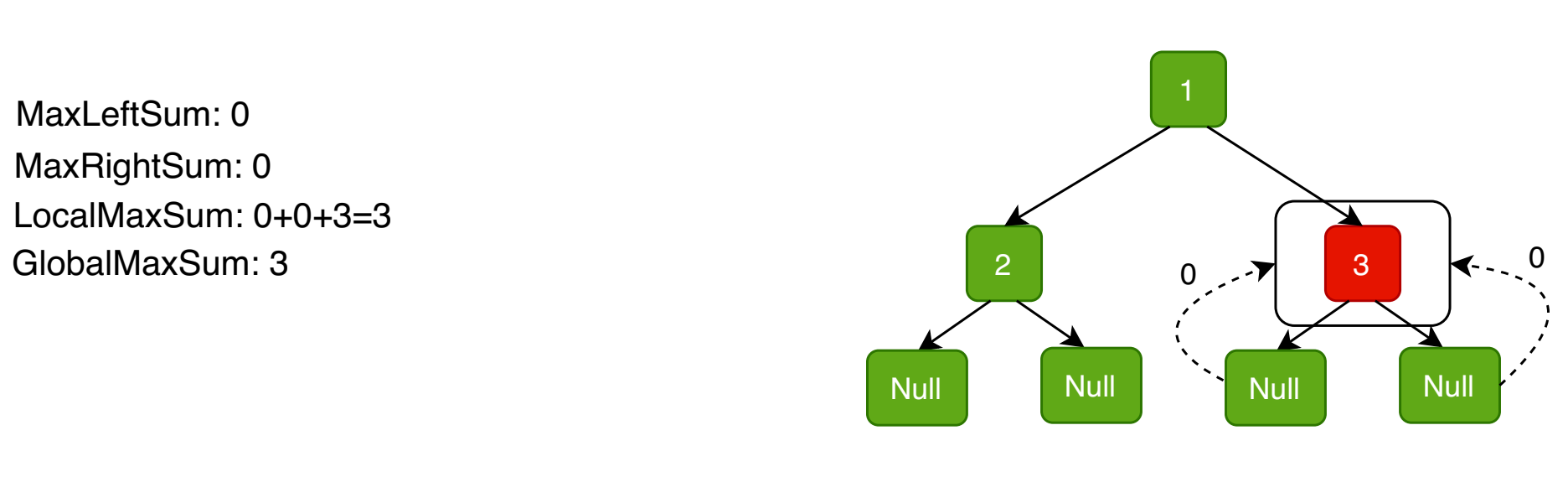


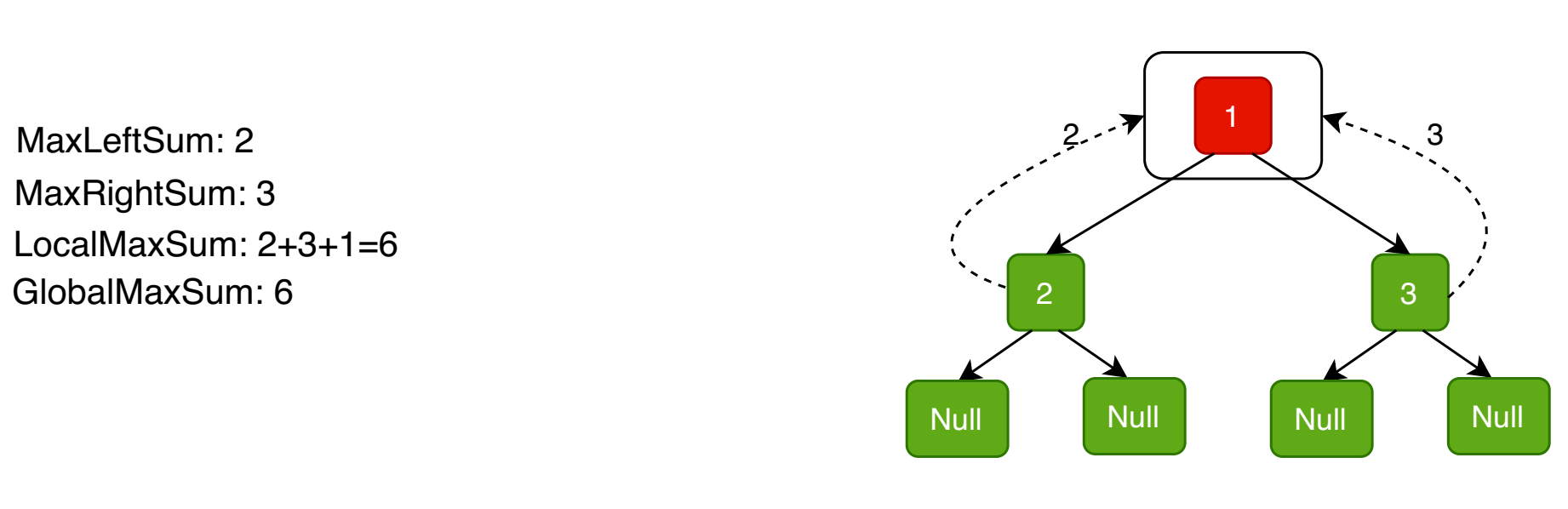


Since exit condition is called so the method will return 0 and hence we will move to the right of the node 3

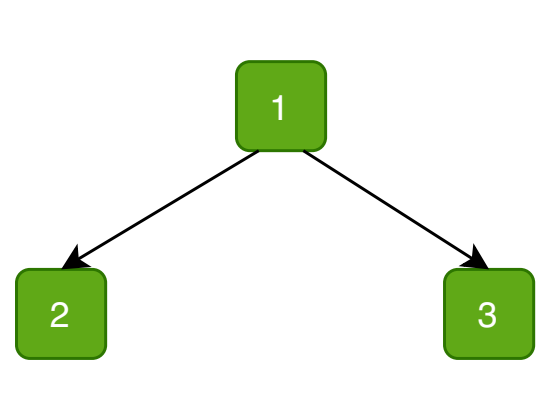






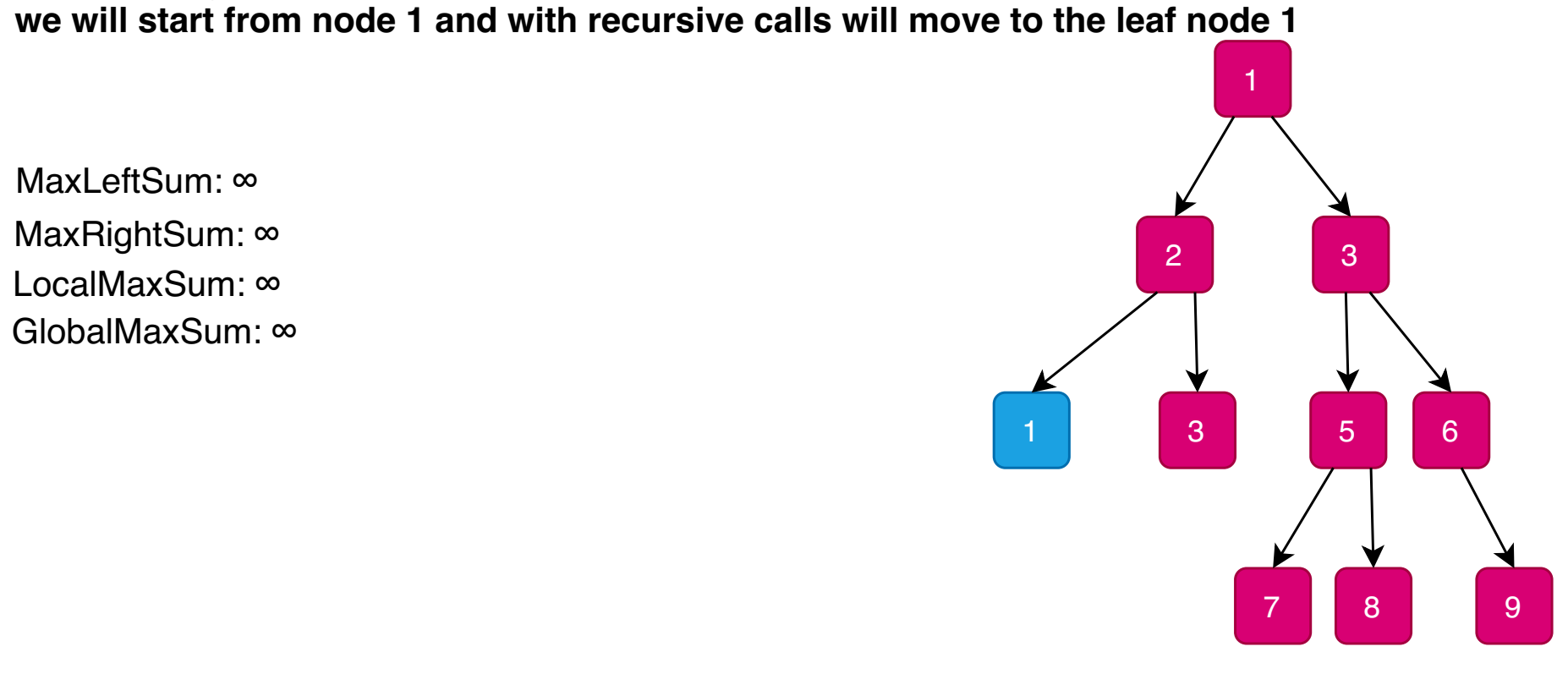


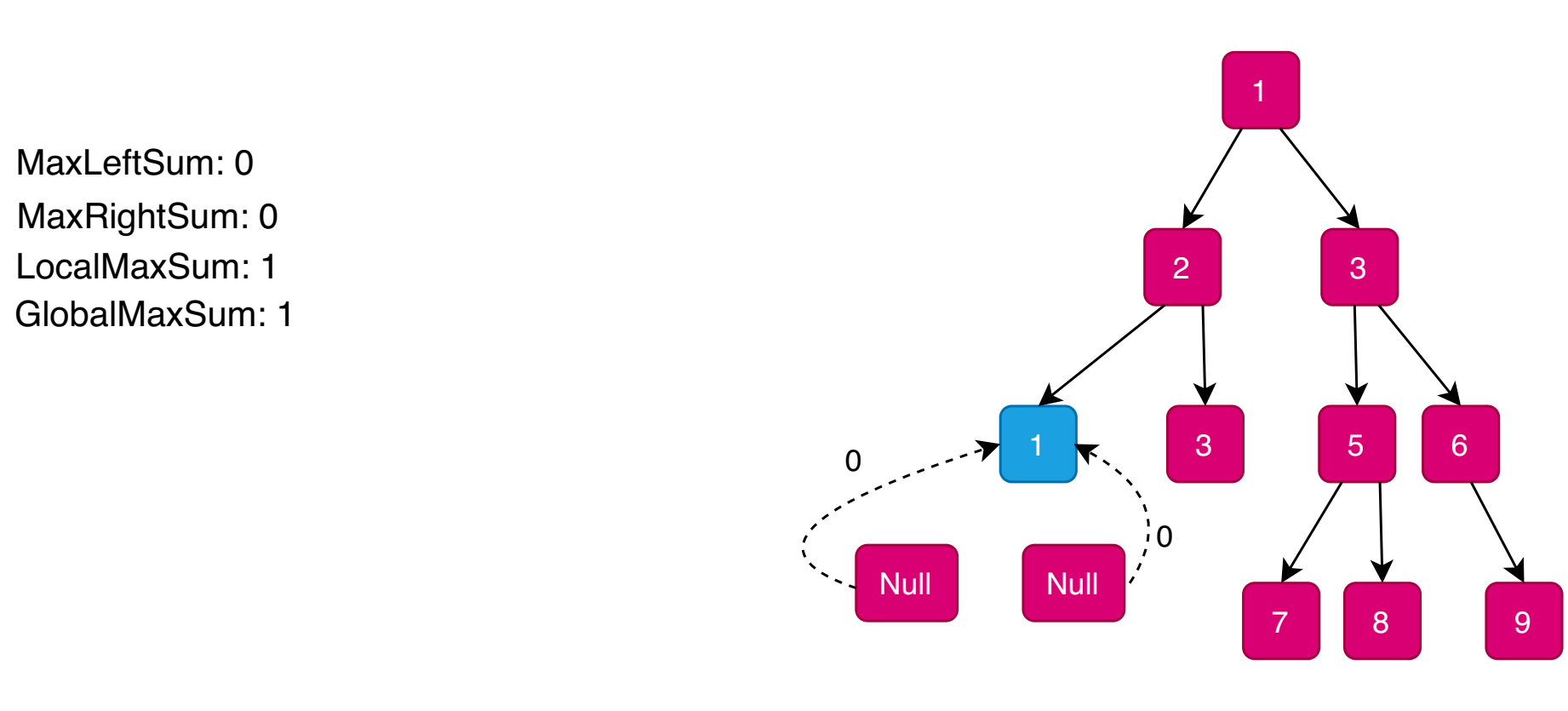
Output: 6

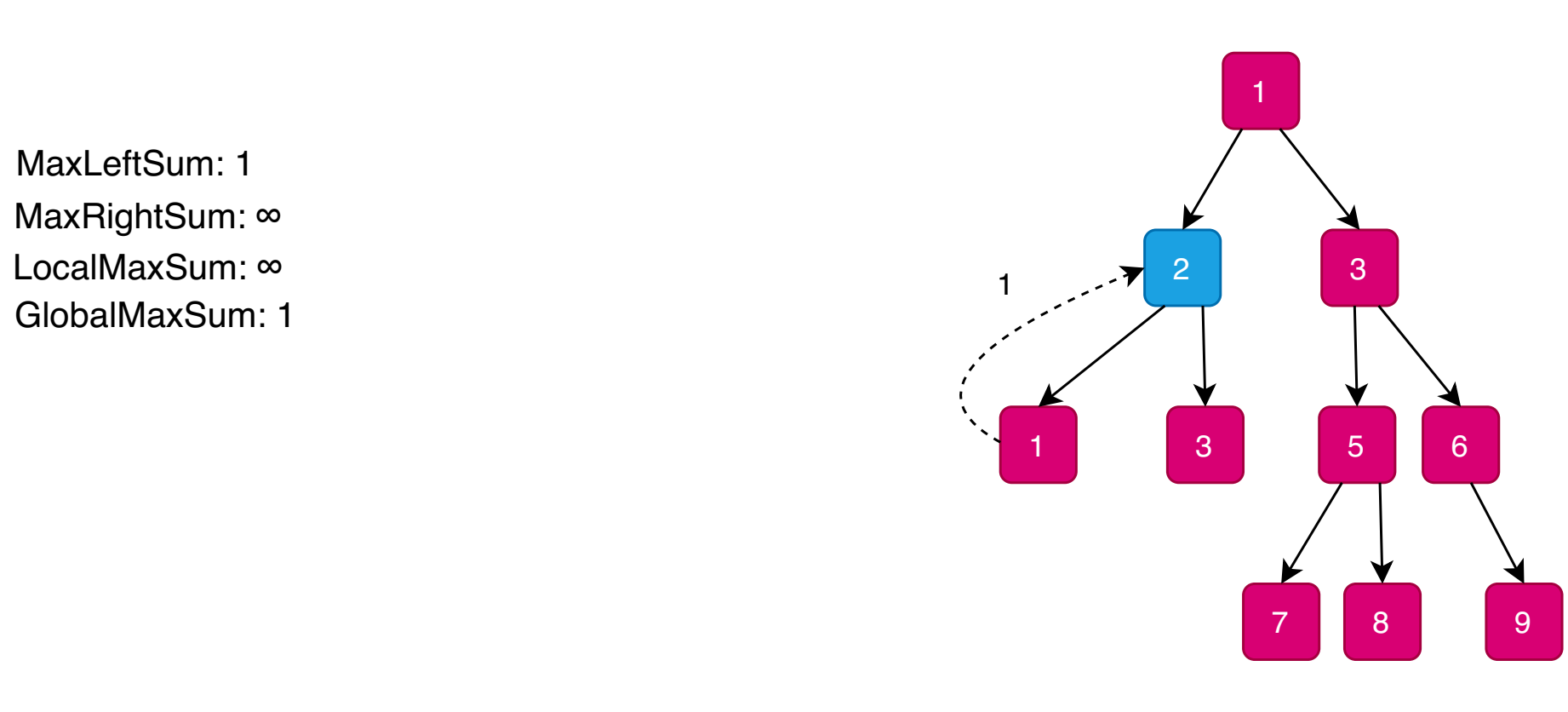


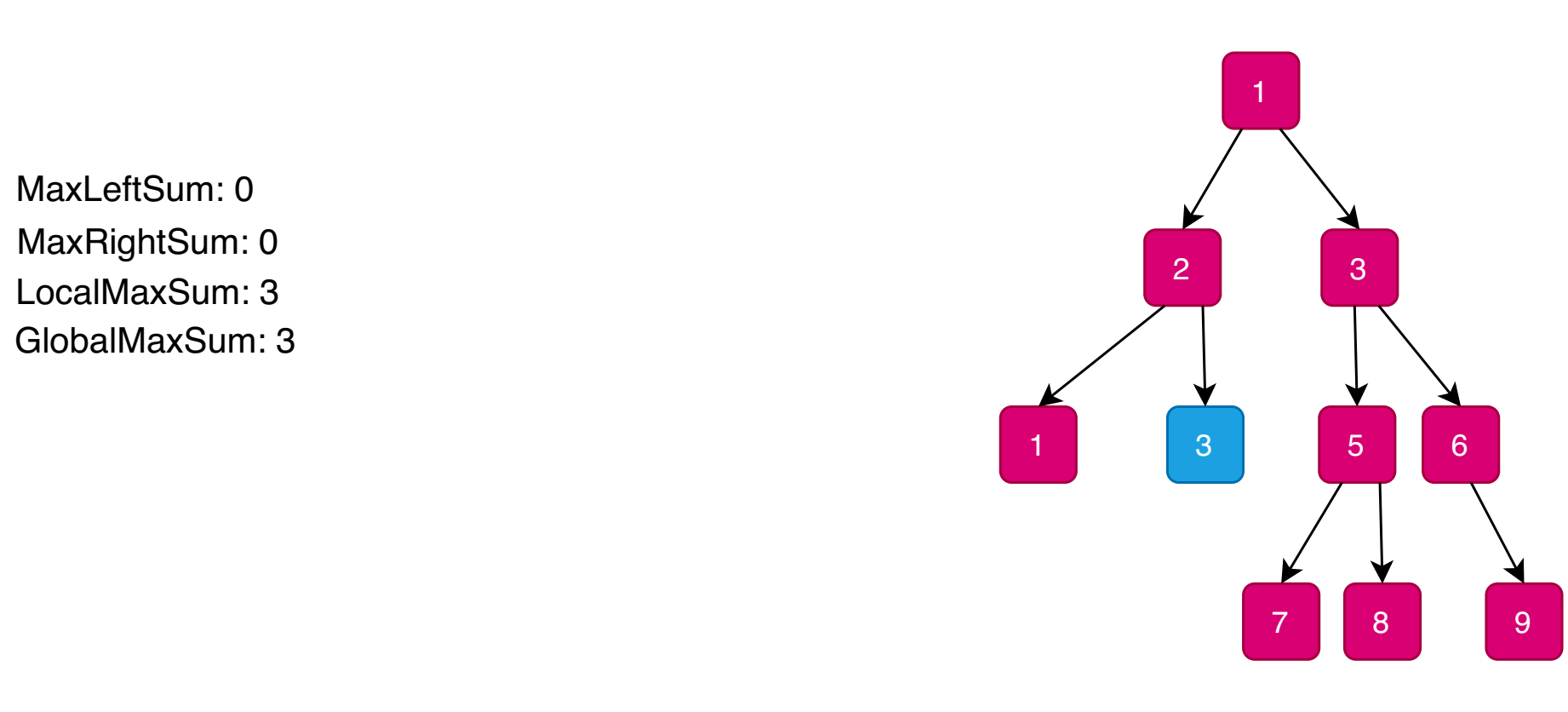
Example: 2

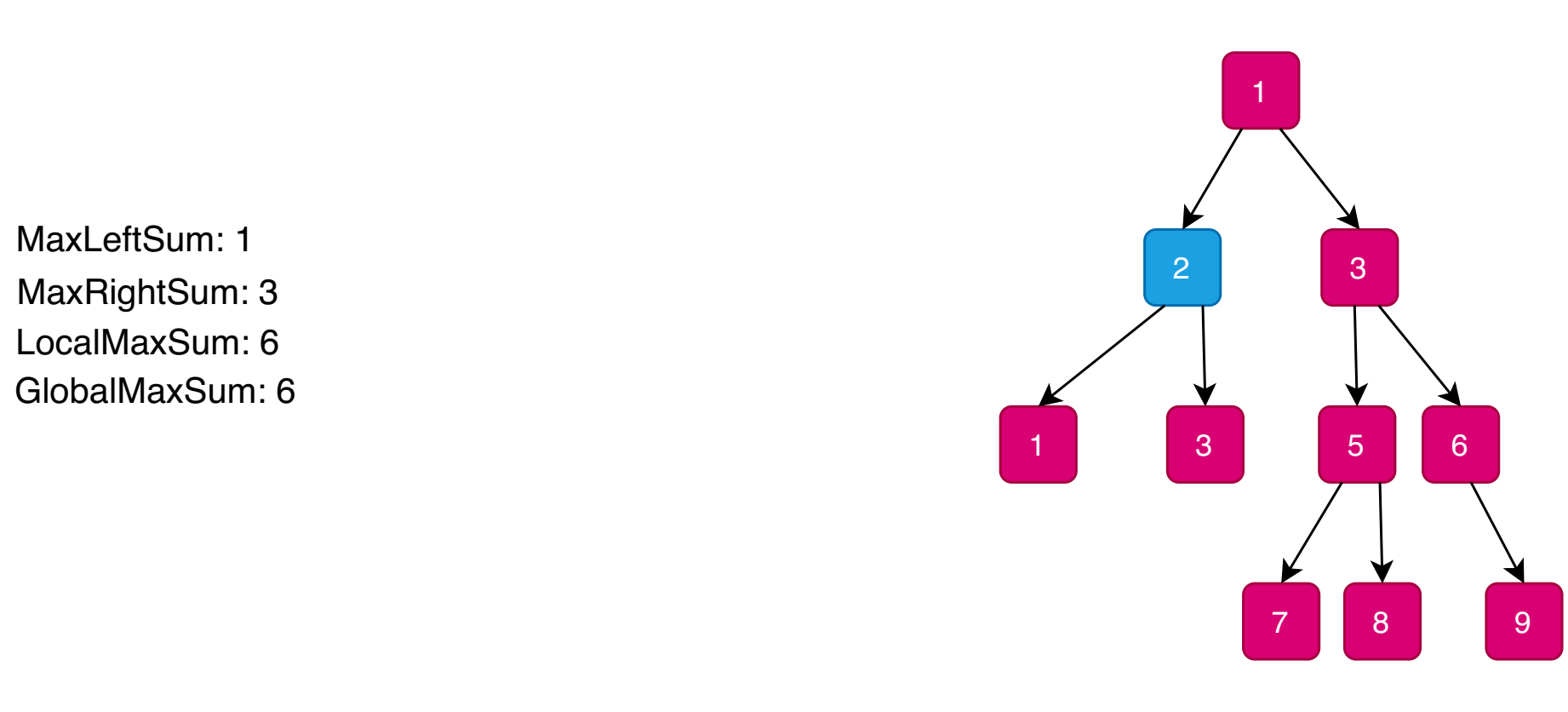
By following the same approach as we have followed in Example1, we will start from node 1 and with recursive calls will move to the leaf node 1



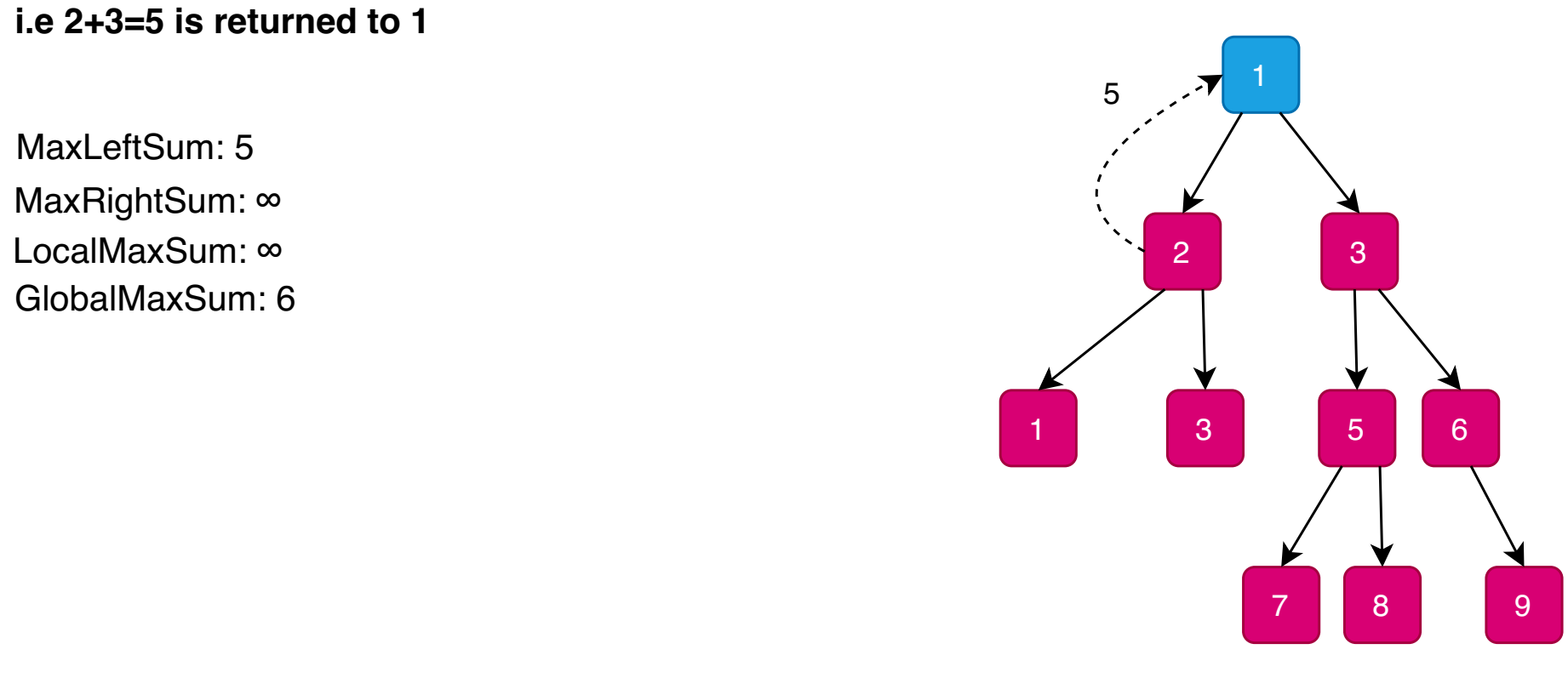


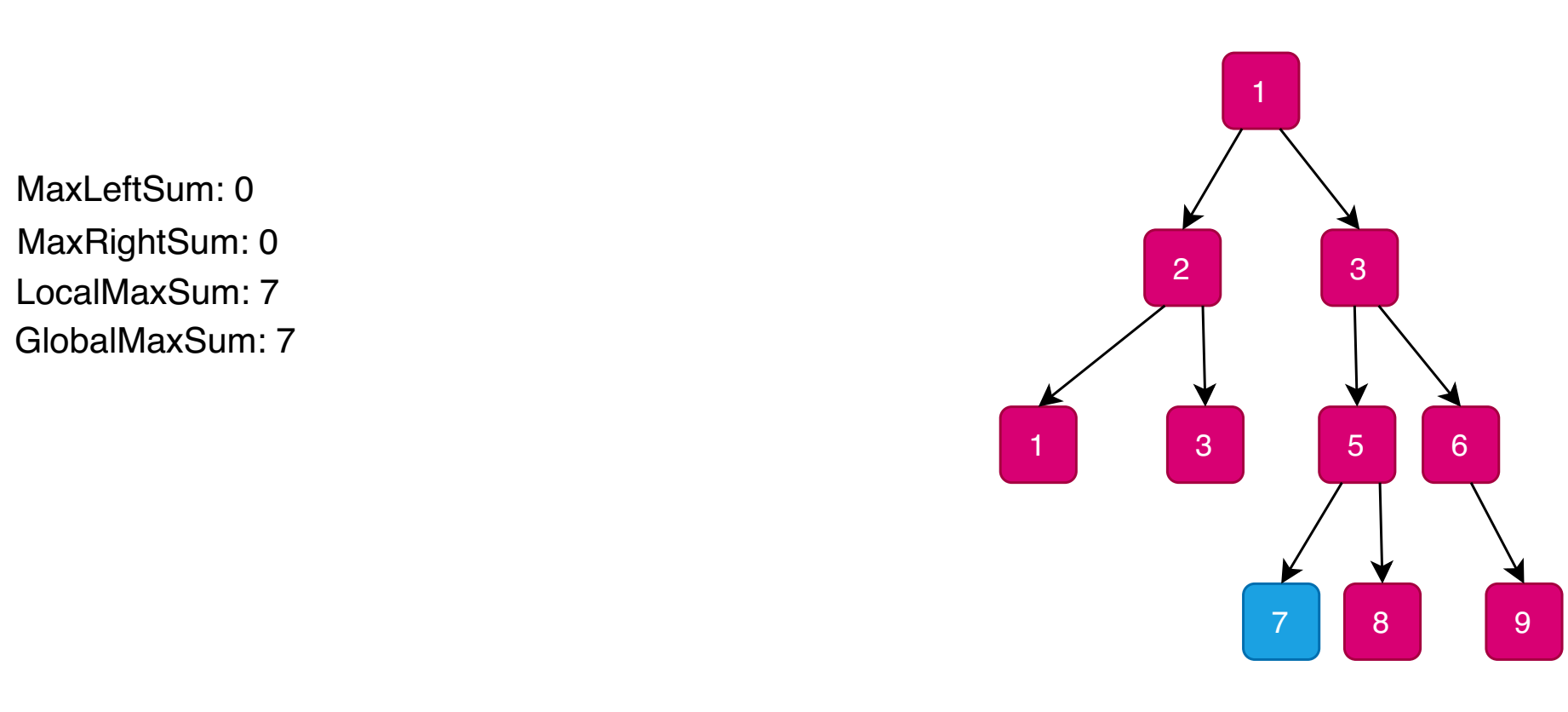


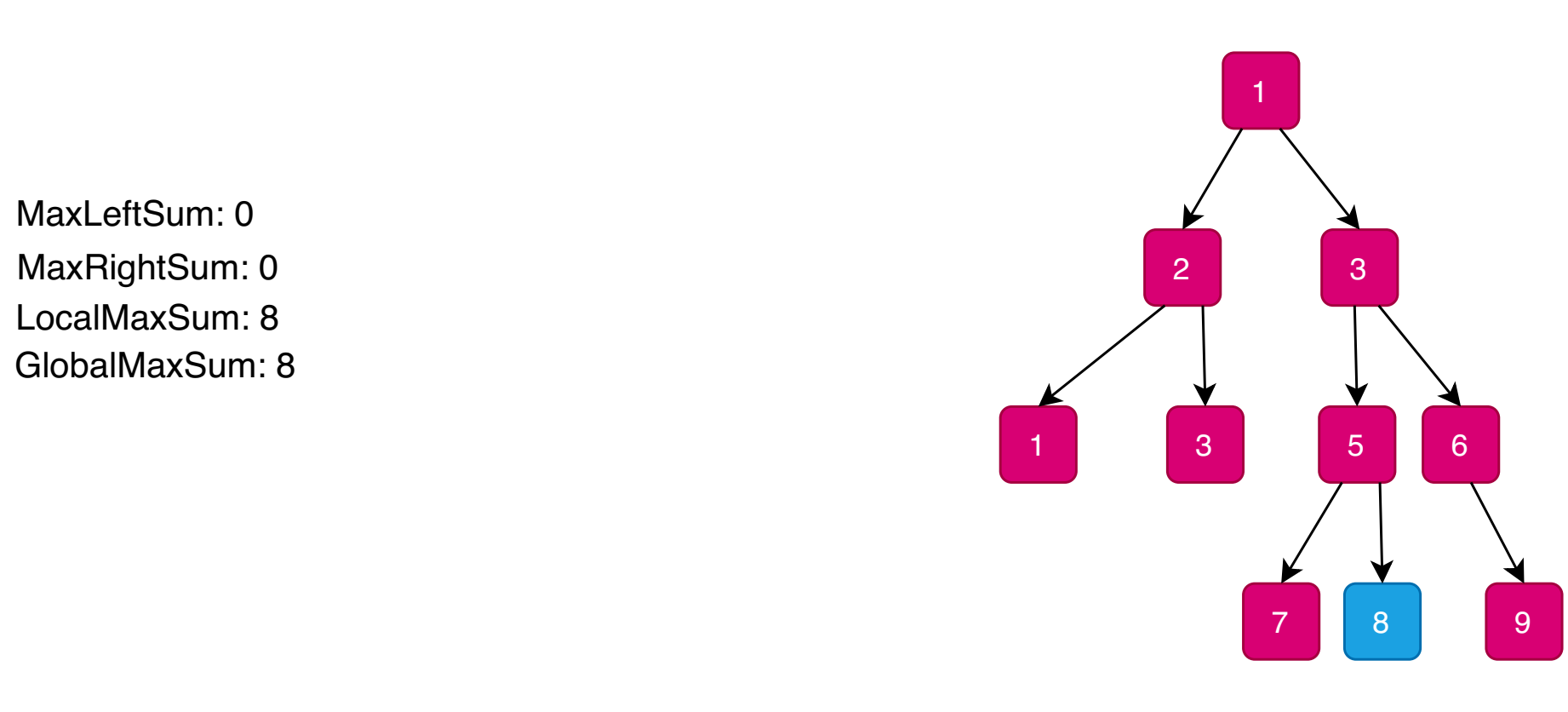


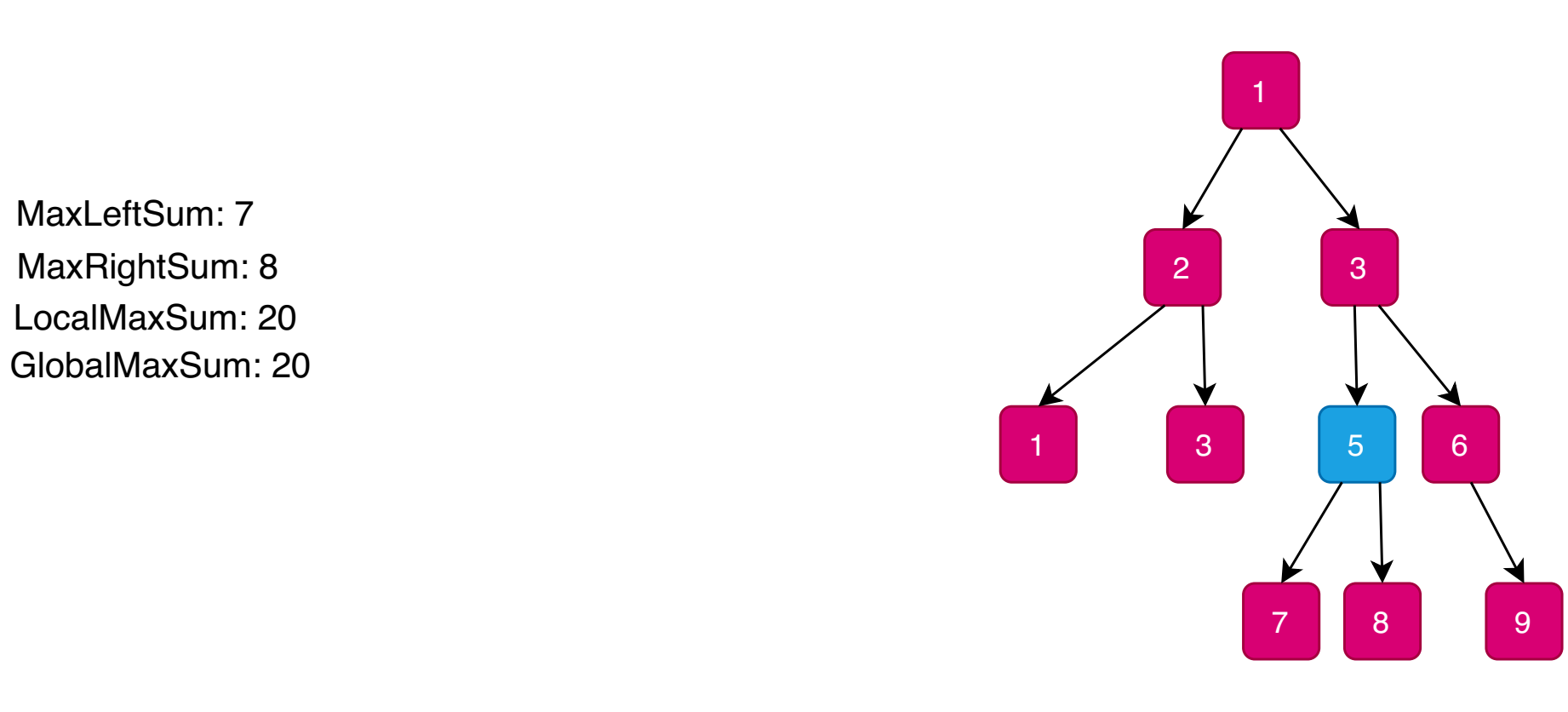


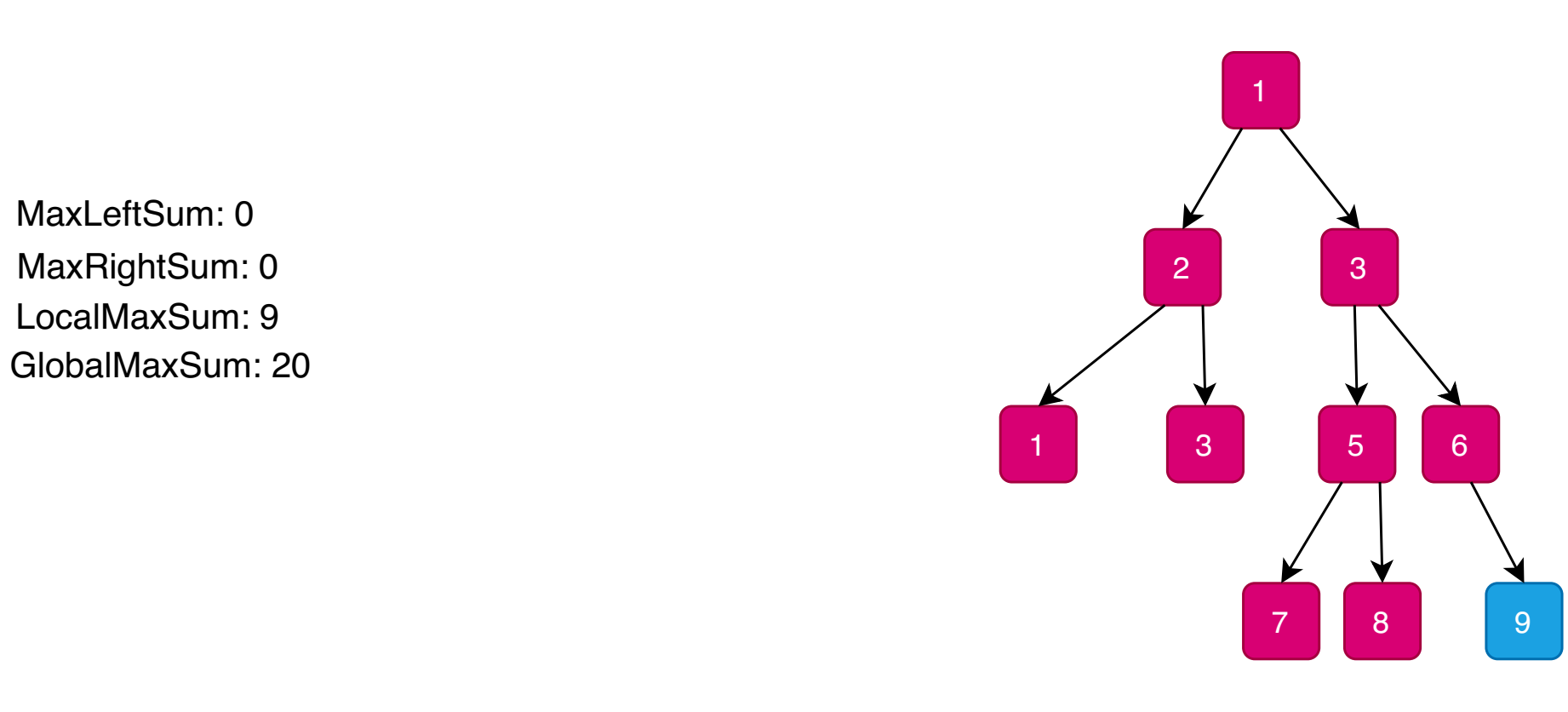
Now, the question is why 5? Actually, we have to return the max. path sum to the respective roots because in that way we can find a path which has the max. sum i.e 2+3=5 is returned to 1

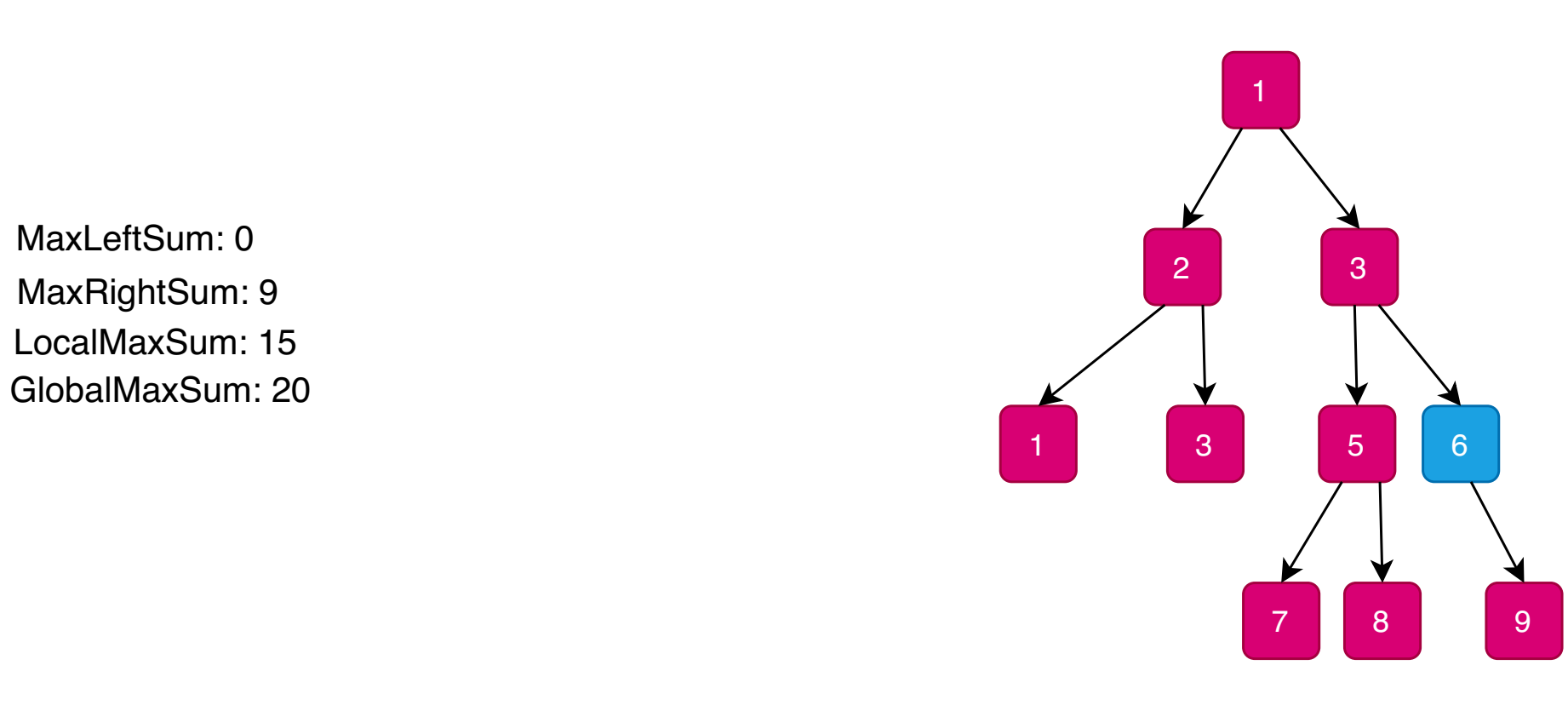


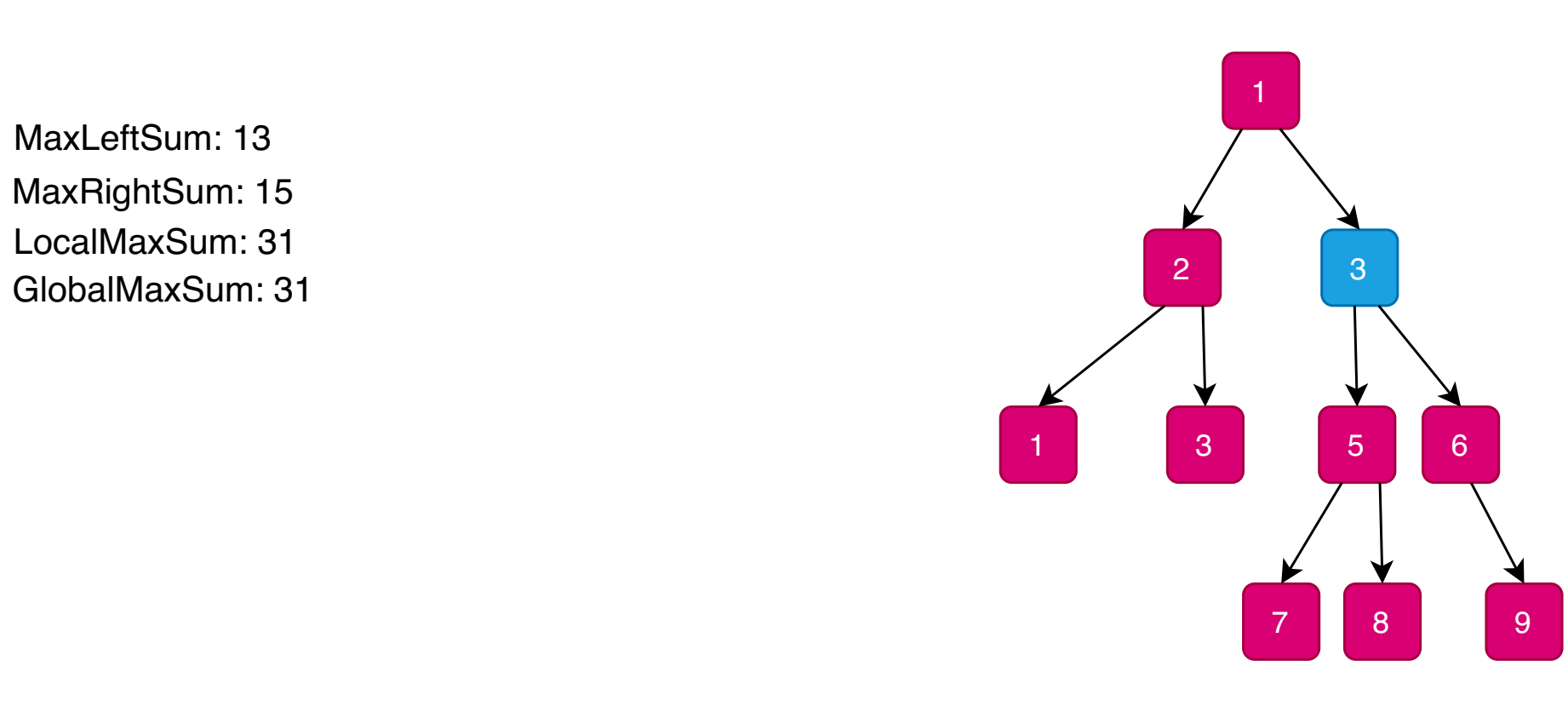


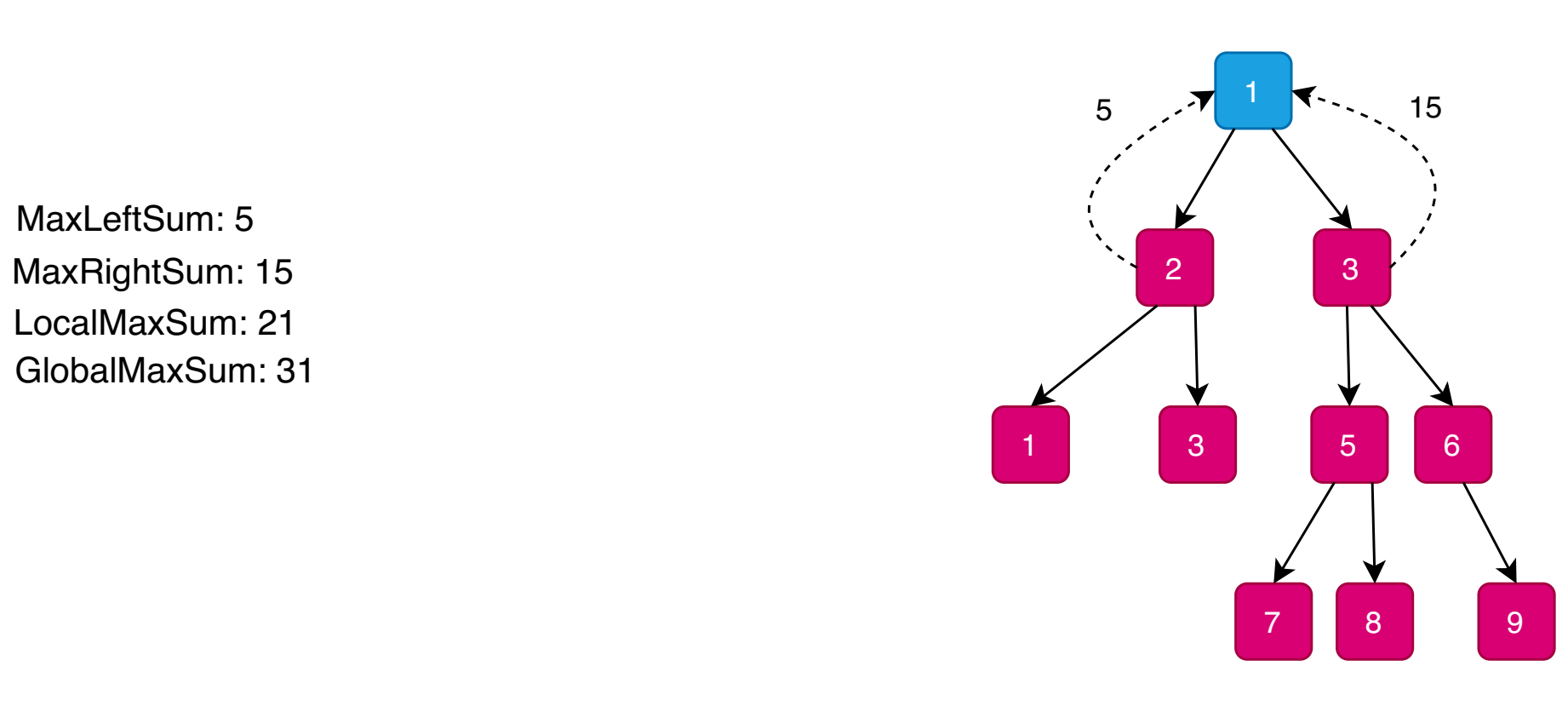




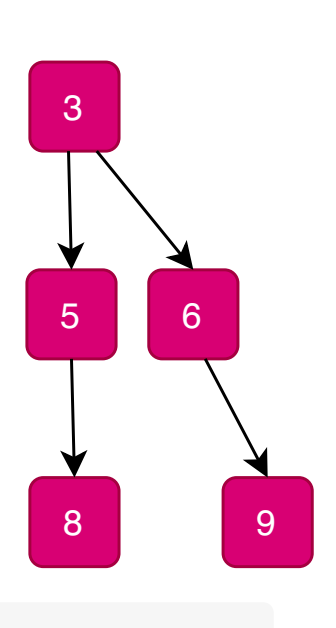








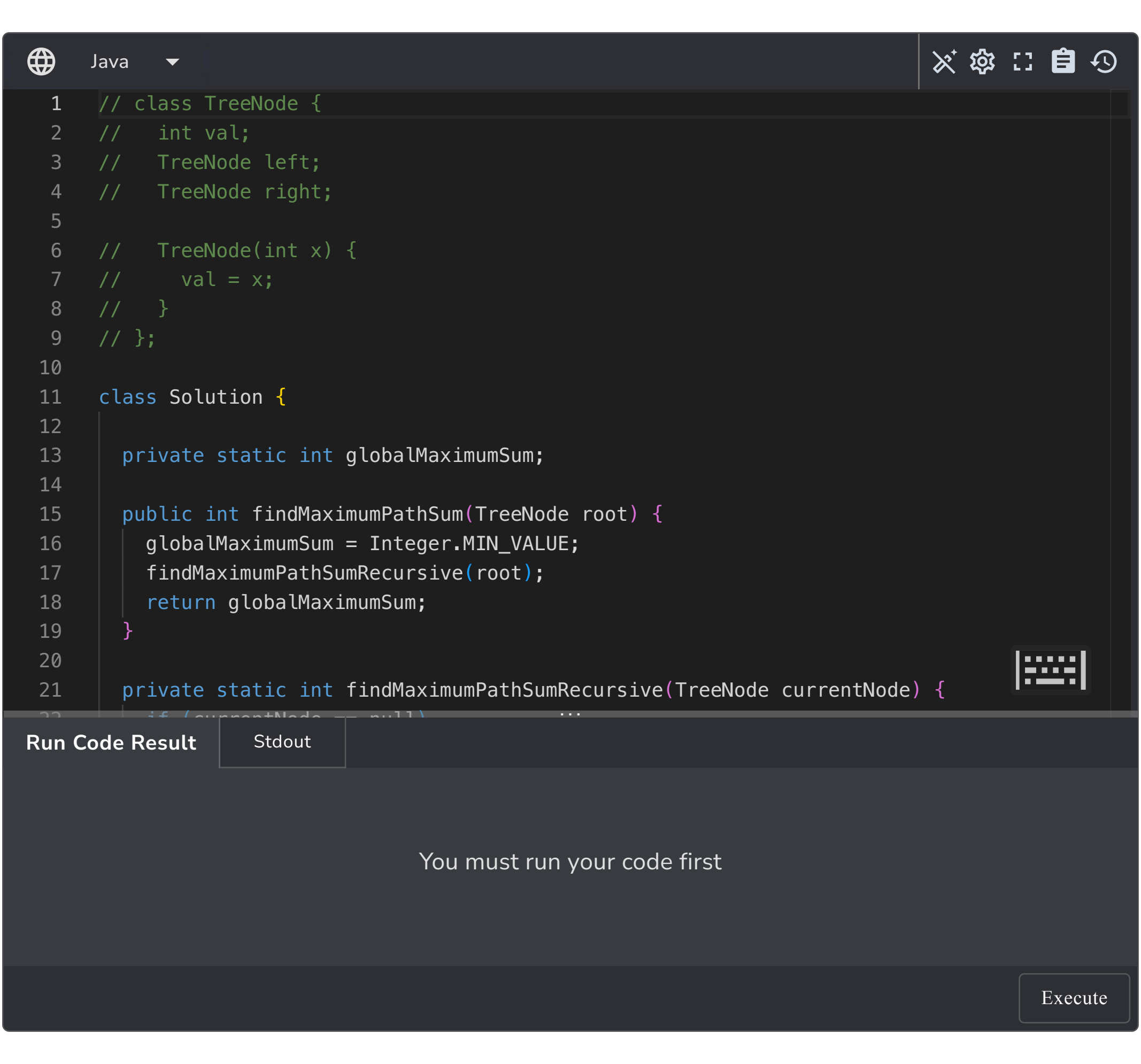
Output: 31



Path with Max Sum

Code

Here is the code for this algorithm:



```java
// class TreeNode {
//   int val;
//   TreeNode left;
//   TreeNode right;

//   TreeNode(int x) {
//     val = x;
//   }
// };

class Solution {

  private static int globalMaximumSum;

  public int findMaximumPathSum(TreeNode root) {
    globalMaximumSum = Integer.MIN_VALUE;
    findMaximumPathSumRecursive(root);
    return globalMaximumSum;
  }

  private static int findMaximumPathSumRecursive(TreeNode currentNode) {
```

You must run your code first

Time Complexity

The time complexity of the above algorithm is $O(N)$, where 'N' is the total number of nodes in the tree. This is due to the fact that we traverse each node once.

Space Complexity

The space complexity of the above algorithm will be $O(N)$ in the worst case. This space will be used to store the recursion stack. The worst case will happen when the given tree is a linked list (i.e., every node has only one child).

← Previous
Problem Challenge 2: Path With Maximum Sum (Hard)

Next →
Introduction To Graph

☐ Mark as Completed